VEHICLE-TO-GRID ENERGY MANAGEMENT USING A HARDWARE PROTOTYPE AND AI ALGORITHMS

PROJECT PRELIMINARY REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF DEGREE OF

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

of

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY

by

ANJANA SURESH (VAS23EC027)

FIDHA CT (VAS23EC036)

KJ SIVARANJINI(VAS23EC044)

MOHAMMED NASEEF VA (LVAS23EC068)



(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Department of Electronics and Communication Engineering

Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)

OCTOBER 2026

Department of Electronics and Communication Engineering
Vidya Academy of Science & Technology

Thalakkottukara, Thrissur - 680 501

(http://www.vidyaacademy.ac.in)

(AN ISO 9001:2015 CERTIFIED INSTITUTION)

Certificate

This is to certify that the preliminary project report titled **"VEHICLE-TO-GRID ENERGY MANAGEMENT USING A HARDWARE PROTOTYPE AND AI ALGORITHMS"**is a bonafide record of the work carried out by **ANJANA SURESH(VAS23EC027),FIDHA CT(VAS23EC036),KJ SIVARANJINI(VAS23EC044),MOHAMMED NASEEF VA(LVAS23EC068)** of Vidya Academy of Science & Technology, Thalakkottukara, Thrissur - 680 501 in partial fulfillment of the requirements for the award of **Degree of Bachelor of Technology** in **Electronics and Communication Engineering** of **APJ Abdul Kalam Technological University**, during the academic year 2026-2027. The Preliminary Project report has been approved as it satisfies the academic requirements in the respect of preliminary project work prescribed for the said degree.

Project Guide

Ms. Honey Mol P.K

Asst. Prof., Dept. of ECE, Senior garde

Head of Department

Dr. Premanand.B

Dept.of ECE

Project Co-guide

Dr. Premanand.B

Prof., Dept. of ECE

Undertaking

We, **ANJANA SURESH(VAS23EC027),FIDHA CT(VAS23EC036),KJ SIVARANJINI(VAS23EC044),MOHAMMED NASEEF VA(LVAS23EC068)** , hereby undertake that the preliminary project work entitled **"VEHICLE-TO-GRID ENERGY MANAGEMENT USING A HARDWARE PROTOTYPE AND AI ALGORITHMS"**, is carried out by me independently under the valuable guidance of **Ms. Honey Mol P.K**, Asst. Prof., and **Dr. Premanand.B**, Prof., Dept. of Electronics and Communication Engineering, Vidya Academy of Science and Technology, Thalakkottukara, Thrissur, in partial fulfillment of the requirements for the award of degree of **Bachelor of Technology in Electronics and Communication Engineering** of **APJ Abdul Kalam Technological University**, during the academic year 2026-2027.

ANJANA SURESH (VAS23EC027)
FIDHA CT (VAS23EC036)
KJ SIVARANJINI(VAS23EC044)
MOHAMMED NASEEF VA (LVAS23EC068)

Thrissur
OCTOBER 2026



Institution Vision

To be a premier institution in engineering education and research, nurturing innovative professionals for sustainable societal development.

Institution Mission

M1: Provide quality engineering education that empowers students with strong technical competence and analytical problem solving abilities.

M2: Promote research, innovation, entrepreneurship and industry collaboration to develop sustainable technological solutions.

M3: Develop ethical and socially responsible professionals with leadership qualities, lifelong learning abilities and a commitment to societal well-being.

Department Vision

To be a centre of excellence in electronics and communication engineering, developing innovative and ethical engineers for sustainable technological development.

Department Mission

M1: Impart quality education with a strong knowledge foundation in core concepts, and complex problem-solving skills in electronics and communication engineering.

M2: Facilitate research, innovation, and industry-oriented practices in emerging areas of electronics, communication, and embedded technologies, fostering sustainable engineering solutions.

M3: Develop professional graduates with leadership, ethical values, teamwork, and lifelong learning abilities, with a commitment to societal well-being.

Department PEOs

PEO-1: Graduates will apply strong technical competencies and problem-solving abilities in electronics and communication engineering to develop sustainable solutions to complex real-world challenges.

PEO-2: Graduates will excel in professional practice as innovators, entrepreneurs, educators, and researchers, applying interdisciplinary knowledge and skills in their careers.

PEO-3: Graduates will demonstrate professional ethics, teamwork, and leadership qualities, contributing responsibly to society and sustainable technological development.

Department PSOs

PSO-1: Analyze and design electronic circuits, VLSI systems, and embedded architectures to develop efficient hardware solutions for engineering applications.

PSO-2: Solve real-world electronics and communication engineering problems using appropriate hardware, software, and modern engineering tools while considering performance, safety, and sustainability.

PSO-3: Innovate in emerging areas of electronics and communication engineering while upholding professional ethics and addressing societal needs.

Department POs

Engineering Graduates will be able to:

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/ develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero car-

bon, culture, society and environment as required. (WK5)

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Acknowledgement

During the course of my preliminary project work several persons collaborated directly and indirectly with me. Without their support it would be impossible for me to finish my work. That is why I wish to dedicate this section to recognize their support.

I want to start expressing my thanks to my project guide, **Ms. Honey Mol P.K**, Asst. Prof., Dept. of ECE,Senior Guide& project Co-guide, **Dr. Premanand.B**, Prof., Dept. of ECE, because of his valuable advice and guidance towards this work. I received motivation, encouragement and hold up from his during the course of work.

I would like to express my sincere gratitude to project coordinators **Mr. Anil M**, Asso. Prof. Dept. of ECE and **Ms. Remya K.R**. Asst. Prof., of ECE department for their valuable suggestion and feedbacks. I am grateful to express my thanks to all the faculty members of our department for their support. I articulate my gratitude to all my friends for their support and help for this work.

I am thankful to **Dr. Premanand.B**, Head of Electronics and Communication Engineering Department, and our Principal **Dr. Sunitha C.**, for their sole co-operation.

Last, but not the least I wish to express my gratitude to God Almighty for His abundant blessings without which this effort would not have been successful.

ANJANA SURESH (VAS23EC027)

FIDHA CT (VAS23EC036)

KJ SIVARANJINI(VAS23EC044)

MOHAMMED NASEEF VA (LVAS23EC068)

Seventh Semester B.Tech (2023 Admissions)

Vidya Academy of Science & Technology

OCTOBER 2026

Thrissur - 680 501.

Abstract

The growing adoption of electric vehicles (EVs) is creating new challenges for existing power systems, particularly in terms of increasing electricity demand and grid stability. Vehicle-to-Grid (V2G) technology offers a promising solution by allowing electric vehicles to act as flexible energy resources that can both draw power from and supply electricity to the grid. The effectiveness of this technology largely depends on the control strategy used to determine when vehicles should charge, remain idle, or discharge energy. The proposed controller uses several key parameters as inputs, including the battery's State of Charge (SOC), grid load, EV power demand, battery temperature, and time. Based on these inputs, the agent selects an appropriate action: charging, idling, or discharging. The reward function is designed primarily to balance grid demand and reduce peak loads. The performance of the proposed DQN-based controller is evaluated using a V2G dataset in a simulated environment. Its effectiveness is compared with three scenarios: a system without V2G support, a rule-based control strategy, and a traditional Q-learning approach. In addition, several machine-learning techniques, including Linear Regression, Decision Tree, Random Forest, XGBoost, and Long Short-Term Memory (LSTM), are examined separately for electricity load prediction and active V2G control. The simulation results illustrate how the DQN agent learns through interaction with the environment and adapts its charging and discharging decisions to different operating conditions. This study focuses on the software development and simulation aspects of the V2G system. Further work will explore battery performance under practical operating conditions and the real-time implementation of the proposed controller using hardware.

Contents

CERTIFICATE

UNDERTAKING

VISION MISSION PEO PSO POs

ACKNOWLEDGEMENT **i**

ABSTRACT **ii**

LIST OF FIGURES **vi**

LIST OF TABLES **vii**

LIST OF SYMBOLS AND ABBREVIATIONS **viii**

1 INTRODUCTION **1**

1.1 Objectives 1

1.2 Motivation 2

1.3 Methodologies 3

1.4 Outline of the Report 4

2 LITERATURE REVIEW **5**

2.1 Paper I:Optimizing demand response and load balancing in smart EV charging networks using AI integrated blockchain framework[1] 5

2.2 Paper II:AI and Machine Learning in V2G Technology: A Review of Bi-directional Converters, Charging Systems, and Control Strategies for Smart Grid Integration [2] 6

2.3 Paper III:Optimizing efficiency of Vehicle-to-Grid system with intelligent management and ANN-PSO algorithm for battery electric vehicles [3] . . . 7
2.4 Paper IV:Priority-Based Vehicle-to-Grid Scheduling for Minimization of Power Grid Load Variance [4] . . . 7

3 SYSTEM METHODOLOGIES 9
3.1 Block Diagram . . . 9
3.1.1 Working of the System . . . 9
3.1.2 Operating Modes . . . 11
3.2 Comparison of Control Approaches . . . 11

4 HARDWARE SETUP OF THE PROJECT 13
4.1 Introduction . . . 13
4.2 Lithium-ion Battery . . . 13
4.3 Microcontroller . . . 14
4.4 Relay Module . . . 15
4.5 OLED Display . . . 16
4.6 Step-down (Buck) Converter . . . 17
4.7 Sensors . . . 18

5 SYSTEM DEVELOPMENT AND IMPLEMENTATION 20
5.1 Introduction . . . 20
5.2 System Development Methodology . . . 21
5.3 Hardware Component Selection . . . 21
5.4 Circuit Design and Hardware Assembly . . . 22
5.5 Sensor Interfacing and Data Acquisition . . . 23
5.5.1 Voltage Measurement . . . 23
5.5.2 Current Measurement . . . 24
5.5.3 Temperature Measurement . . . 24
5.6 ESP32 Programming and Control Logic Development . . . 24
5.7 OLED Display Integration . . . 25
5.8 Load Testing Using a DC Fan . . . 25
5.9 Integration of Hardware and Software . . . 25

5.10 Testing and Performance Evaluation 26
5.11 Prototype Development and Final Assembly 26
5.12 System Operation 28
5.13 Summary 28

6 PROJECT STATUS COMPLETED IN S7 29
6.1 Hardware Prototype Development 29
6.1.1 Integration of ESP32 29
6.1.2 Hardware Testing and Initial Verification 30
6.2 Simulation and Algorithm Comparison 30
6.2.1 DQN Training Convergence and Reward Improvement 30
6.2.2 Comparison plot 31

7 PROJECT TO BE COMPLETED IN S8 33
7.1 Introduction 33
7.2 Study and Analysis of DDPG and PPO Algorithms 34
7.2.1 Deep Deterministic Policy Gradient (DDPG) 34
7.2.2 Proximal Policy Optimization (PPO) 34
7.2.3 Plan for the comparison 35
7.3 Dashboard Making and Displaying 35
7.4 Peak Shaving Using Hardware 36
7.5 Final Testing, Validation and Project Demonstration 36
7.6 Expected Outcome 37
7.7 Mapping with POs and PSOs 38
7.8 Mapping with SDGs 39

BIBLIOGRAPHY 40

APPENDIX 41

List of Figures

3.1 Block Diagram 10

4.1 ESP32 15
4.2 Relay Module 15
4.3 OLED Display 16
4.4 Converter 17
4.5 Voltage Sensor 18
4.6 Temperature Sensor 19

5.1 Circuit Diagram of the Proposed V2G System. 23
5.2 Developed Hardware Prototype 27

6.1 Mean Reward Progression During DQN Training 31
6.2 V2G Load Control–DQN vs Q-Learning vs Rule-Based 32

List of Tables

3.1 Qualitative comparison of V2G decision approaches 12

7.1 Mapping of the project with POs and PSOs 38

7.2 Mapping of the project with SDGs 39

List of Symbols and Abbreviations

ABCD	Use in alphabetical order
DQN	Deep Q-Network
LSTM	Long Short Term Memory
RL	Reinforcement Learning
SoC	State of Charge
V2G	Vehicle to Grid
LSTM	Long Short Term Memory
RL	Reinforcement Learning

Chapter 1

INTRODUCTION

1.1 Objectives

Electric vehicles are no longer just a way to get from one place to another; with Vehicle-to-Grid technology, they can also become a source of power for the grid. This project aims to bring that idea to life by building a safe, low-voltage (12V) prototype using an ESP32, where a battery can both draw energy from a simulated grid and give it back when needed. At the heart of the system is a simple belief: charging and discharging should be a smart decision, not a fixed rule. To make that possible, the prototype keeps a constant watch on the battery's voltage, current and temperature, and works out its State of Charge so it always knows how full and how healthy the battery is. Using this information, a learning-based controller, compared across several machine learning models and a DQN, chooses whether the battery should charge, discharge or stay idle, depending on the battery's condition and the demand on the grid. By avoiding unnecessary charge and discharge cycles, the system makes better use of the battery's energy and helps protect its life. During peak hours it supports the grid by supplying power, and during low-demand periods it recharges, easing the pressure on the supply. Finally, the project is designed as an affordable and easy-to-replicate platform, so that students and researchers can use it to explore and build on AI-based smart grid and V2G ideas.

1.2 Motivation

These days it is hard to ignore how quickly electric vehicles are spreading. Every EV that plugs in adds to the load on the grid, and in the evening, when people get back from work and charge their cars together, the grid is already at its busiest. Yet the batteries in those parked cars are just sitting there, full of energy that nobody is using. That mismatch, a grid under pressure on one side and thousands of idle batteries on the other, is what first got us interested in this project. Vehicle-to-Grid (V2G) seemed like a good way to put that idle energy to work. But when we started reading about it, we found that most V2G systems still run on fixed rules. They charge below one voltage level and discharge above another, and that is about it. If the battery condition changes or the demand on the grid shifts, the system does not notice. So battery capacity is not used well during peak hours, and the battery goes through extra charging and discharging cycles that wear it out sooner. We kept thinking that a system which looks at what is actually happening and then decides would make much more sense. The papers we went through in our literature survey pointed the same way. A lot of them stayed at the theory or simulation level, and some had very little hardware testing. Others spent most of their effort on the converter circuit and said little about how the decisions are made. We saw a gap there for a project that does both: a real hardware prototype with a smarter controller behind it. That is where Artificial Intelligence came in. A machine learning model can take in the battery voltage, current, temperature, State of Charge and whether the grid is in a peak or off-peak period, and then pick between charging, discharging or staying idle. And since a small model can run on a cheap microcontroller like the ESP32, the whole idea stays practical and low cost. Safety also played a big part in our decision. Working with real mains voltage is risky and expensive for a student project. With a 12V Li-ion battery and a simulated 12V grid, we can try out the full idea, watch how it behaves and fix our mistakes without any danger. Lastly, we hoped this work would be useful to others too. A simple, affordable setup like this can help students and researchers explore AI-assisted smart grids and V2G, and it can be extended later with solar power, battery health prediction or IoT-based remote monitoring. All of this together is why we chose to take up this project.

1.3 Methodologies

When we started this project, we knew we could not just connect a battery and hope for the best. So we took it one step at a time, and each step helped us understand the next one better. First, we read a lot. We went through research papers on bidirectional EV chargers and V2G energy management to find out what others had done and what they had missed. We noticed that many works were only simulations, and most of the control was based on fixed rules. That gave us a clear idea of what our system should do: monitor the battery, make smart decisions, and act on them safely. Then we moved on to the design. We chose a 12V Li-ion battery, an ESP32 as the brain of the system, sensors for voltage, current and temperature, a relay/MOSFET circuit for switching, and an OLED display to show what is happening. A 12V DC supply acts as our grid, and a 12V DC load such as an LED bulb or a fan represents the power demand. Before building anything, we simulated the design in MATLAB/Simulink and Proteus, just to be sure the idea worked on paper. The AI part came next. Real V2G data is hard to get, so we prepared a synthetic dataset in CSV format. It includes battery voltage, current, temperature, State of Charge (SOC) and peak or off-peak demand, along with the right decision for each case: charge, discharge or stay idle. In Python, we trained and compared several models. Linear Regression, Decision Tree, Random Forest, XGBoost and LSTM were used to predict grid load, while Q-Learning and a Deep Q-Network (DQN) were tried for control. We plotted how each one behaved over a 24-hour cycle and used that to decide which approach suits our system best. The final model is then made small using TinyML so it can run on the ESP32 itself. In the hardware, the working is simple. The sensors read the battery, and the ESP32 calculates the SOC and passes the values to the AI model. The model suggests a mode, and the ESP32 switches the relay/MOSFET circuit to charge the battery, discharge it to the load, or keep it idle. The OLED display shows the voltage, current, SOC and mode in real time. We are currently setting up this hardware. Finally, we will test the whole system under different battery and demand conditions, see where it goes wrong, and fix and fine-tune it until it behaves the way we want.

1.4 Outline of the Report

This project report is organised into six chapters, which present the design, development and testing of the RL-based SOH prediction system in a step-by-step manner.

Chapter 1 – Introduction: This chapter introduces the importance of battery health in electric vehicles and explains why accurate SOH prediction is needed. It also presents the objectives, motivation and methodology of the project, and gives a short outline of the report.

Chapter 2 – Literature Review: This chapter discusses earlier research related to battery SOH estimation, including model-based, machine learning and deep learning approaches. It describes the findings of the papers reviewed, points out their limitations, and explains why a reinforcement learning approach is worth exploring.

Chapter 3 – System Methodologies: This chapter explains the methods followed in the project. It describes the existing and proposed systems, along with the block diagram and the flowchart of the work. It also covers the dataset and the data preparation steps, and the software tools used, such as Python and MATLAB/Simulink.

Chapter 4 – System Development: This chapter explains how the system was developed. It introduces the overall design and describes each important component in turn, including the lithium-ion battery, the ESP32, the OLED display, the relay module, the step-down converter, and the voltage, current and temperature sensors. It also explains what each component does and why it was chosen.

Chapter 5 – Hardware Setup of the Project: This chapter presents the practical test setup. It shows how the components are connected on the prototype board and how the control part works, including how the ESP32 measures the battery, protects it from overcharge and deep discharge, and records the data used for SOH estimation.

Chapter 6 – Project Status Completed in S7: This chapter summarises the work finished in the seventh semester. It reports the progress made so far, such as the literature review, the system design, the hardware setup and the data collection, and shows what has been achieved against the project plan.

Chapter 7 – Project Status to be Completed in S8: This chapter describes the work planned for the eighth semester. It lists the remaining tasks, such as training and testing the RL model, building the solid-state battery model

Chapter 2

LITERATURE REVIEW

This chapter reviews previous research on Vehicle-to-Grid (V2G) technology, focusing on bidirectional power flow between electric vehicles and the electrical grid. It highlights the methods, key findings, and limitations of existing studies to identify the research gap and establish the foundation for the proposed V2G system.

2.1 Paper I:Optimizing demand response and load balancing in smart EV charging networks using AI integrated blockchain framework[1]

When more and more EVs plug in, the charging network faces three worries. Too many cars charging at the same time overloads the grid. A central server that controls everything becomes a single weak point, so data privacy and security are at risk. And the system must keep working when the number of EVs grows. Singh et al. asked how an EV charging network can balance its load, protect its data and still scale up. Method and conclusion. The authors proposed a framework called DR-LB-AI. Its AI part predicts the demand and spreads the charging load, and it encourages charging during off-peak hours. Its blockchain part keeps the energy transactions secure and tamper-proof, without depending on one central server. The paper reports that the framework reduced peak demand stress by 20 percent, and it also reports improvements of 98.43 percent in scalability and 97.71 percent in security. The AI was trained on a public EV charging dataset from Kaggle. The authors list limitations in future work: the framework is yet to be tested

under varied grid conditions, and it has not yet included solar and wind power. The paper also notes that such systems need a strong infrastructure. This is relevant to our project because it supports AI-based load balancing and off-peak charging. Our project differs in that a DQN controller makes the charge, idle and discharge decisions, and we test it on a real low-voltage hardware prototype.

2.2 Paper II:AI and Machine Learning in V2G Technology: A Review of Bi-directional Converters, Charging Systems, and Control Strategies for Smart Grid Integration [2]

V2G system is more than a battery and a plug. It needs a bidirectional converter to move power in both directions, a charger that follows safety standards, and a controller that decides when to charge or discharge. The difficulty is that traditional controllers such as PID are tuned for fixed conditions, while the grid, the user and the battery keep changing. The question is whether machine learning can make these decisions adaptively. Munusamy and Vairavasundaram reviewed the field to find out. The review compares converter topologies, charging standards, battery behaviour and safety measures, and then goes through control methods: PID, model predictive control, fuzzy logic, neural networks, and supervised, unsupervised and reinforcement learning. It reports that supervised learning dominates power-converter applications at about 91 percent, while reinforcement learning is only about 5 percent. It also notes that classifying the battery state into charge, discharge and idle is a natural machine-learning task, and that frequent V2G cycling can speed up battery degradation. The authors point out that real V2G systems need smart meters, two-way controllers and strong cybersecurity. This supports our choice of a learning-based controller with SOC and temperature monitoring, and a low-voltage prototype with safe switching.

2.3 Paper III:Optimizing efficiency of Vehicle-to-Grid system with intelligent management and ANN-PSO algorithm for battery electric vehicles [3]

Solar and wind power are growing fast, but their output keeps changing with the weather, and that makes the grid harder to run. EV batteries can help by storing energy and giving it back, but this works only if charging and discharging are managed carefully. Without management, an EV can be overcharged, or the grid has to cover the shortage, which costs more. Nouri et al. therefore asked how a V2G system can use EV batteries intelligently while taking the most energy from a solar source. Method and conclusion. The authors built a system where solar panels, EV batteries, a three-phase load and the grid are linked through converters. A neural network combined with particle swarm optimization (ANN-PSO) pulls the maximum power from the solar panels, even when sunlight changes suddenly. A fuzzy logic controller then decides, using set conditions, when the EV battery should charge or discharge, and it protects the battery from overcharging. The simulations showed stable operation without oscillation and a steady energy efficiency of about 97 percent. The efficiency dropped when the EVs did not take part, which shows why V2G helps. The authors admit that the work is simulation-based and that real-world testing is needed. This is relevant to our project because it shows that intelligent charge and discharge control improves V2G efficiency. Our DQN controller goes a step further, since it learns its decisions instead of following fuzzy rules fixed in advance.

2.4 Paper IV:Priority-Based Vehicle-to-Grid Scheduling for Minimization of Power Grid Load Variance [4]

When many EVs plug in at the same time, they behave like a big extra load on the grid. They can create a new evening peak, overload transformers and cables, and disturb the voltage. But the same batteries can help the grid if they are allowed to give power back. The difficulty is that most earlier V2G studies looked only at the benefit to the utility, or only at the benefit to the EV owner. A driver will not join V2G if it risks leaving the battery too empty to drive. So the practical question is how to flatten the grid's load

curve without ignoring the charge level of each vehicle. Hashim et al. answered with a priority-based scheduling algorithm. The algorithm compares the grid load with two target levels and works in three modes. When the load is below the minimum target, it fills the valley by charging connected EVs. When the load is between the two targets, it only charges EVs with low charge. When the load is above the maximum target, it shaves the peak by discharging EVs that have enough charge. The key idea is the battery's state of charge (SOC). EVs below 55 percent SOC get higher charging power and are never asked to discharge, while EVs with high SOC discharge more. Charging stops at 90 percent to protect the battery. The method was tested on a simulated area with 1300 EVs, 850 homes and shops, and 85 percent charger efficiency. The peak load fell from 7.3 MW to 6 MW (17 percent), the lowest night load rose from 2.3 MW to 4.5 MW, and the gap between peak and off-peak dropped from 5 MW to 1.5 MW. The authors admit that the study is simulation-based. Some hours were not fully optimized because most EVs were already charged, and the 55 percent and 90 percent limits are fixed rules. This is relevant to our project because it gives us a rule-based baseline with SOC thresholds and peak/off-peak modes. Our AI controller can be compared against it, and it can learn the thresholds instead of using fixed values.

Chapter 3

SYSTEM METHODOLOGIES

3.1 Block Diagram

The block diagram in Figure gives a simple picture of our V2G prototype. We built it on a small and safe scale, where a 12V battery pack stands in for the EV and the main power supply stands in for the grid. Energy flows from the supply through a 12V adapter and a DC-DC converter into the battery during charging. In V2G mode, it flows back out of the battery through the converters and the step-up transformer to the bulbs and the fan, which represent the demand on the grid. The Battery Management System (BMS) keeps the battery safe. The ESP32 controller is the brain of the system. It takes the user switch input and the battery status from the BMS, controls the MOSFET and motor, and shows the system state through the green and red LEDs. In simple terms, the BMS protects the battery, the converters move the energy, and the ESP32 decides when and where it goes.

3.1.1 Working of the System

Power source: The main power supply feeds a 12 V adapter, which converts it into the steady 12 V DC that the rest of the system works with. This block plays the part of the grid in our prototype.

DC–DC converter and BMS (charging and V2G): The 12 V from the adapter goes to the DC–DC converter, which adjusts the voltage to the level needed for the battery. The converter is connected to the Battery Management System (BMS) by a two-way arrow marked “Charge / V2G”, because energy can travel in both directions here. When the system is charging, power flows from the converter, through the BMS, into the battery.

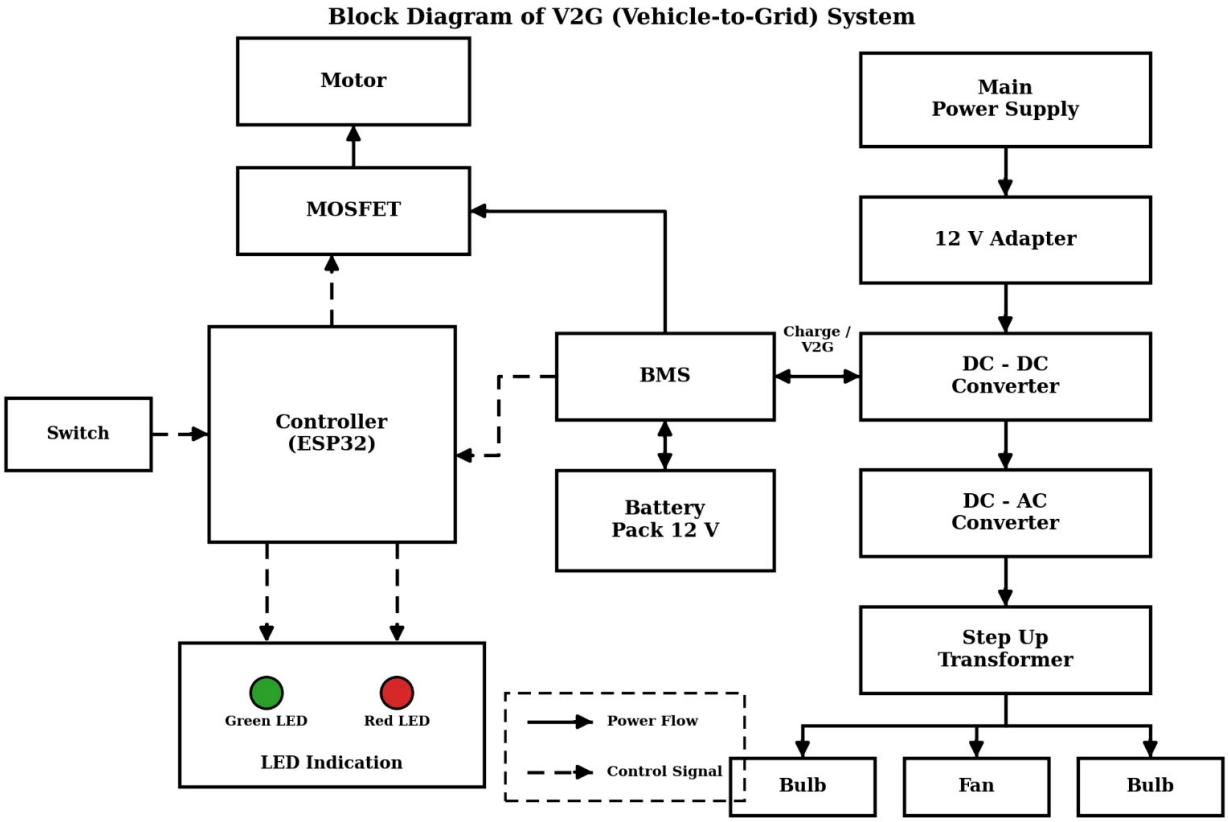


Figure 3.1: Block Diagram

When the system is supporting the load, power flows the other way, from the battery through the BMS to the converter. The BMS stands guard over the battery in both cases. It protects the cells against overcharge, deep discharge and short circuits, and it passes the battery status to the controller through the dashed line.

Battery pack: The 12 V battery pack is the energy store that represents the EV battery. It takes in energy when demand is low and gives it back when demand is high. It is connected to the BMS by a two-way arrow because it both charges and discharges.

Inverter, transformer and loads: The output of the DC–DC converter also goes forward to the DC–AC converter, which changes the DC power into AC. A step-up transformer then raises the AC voltage to the level needed by the loads, which are the two bulbs and the fan shown at the bottom of the diagram. These loads represent the demand on the grid. Because the converter is fed either from the adapter or from the battery, the loads can be powered from the grid or supported by the battery, which is the main idea of Vehicle-to-Grid.

Motor branch: A separate branch runs from the BMS to a MOSFET and then to a motor.

The MOSFET acts as an electronic switch that lets the battery run the motor, which represents the drive of the vehicle. The motor is not driven directly by the controller; the ESP32 only sends a small control signal to the MOSFET, and the MOSFET handles the larger current.

Controller (ESP32): The ESP32 is the brain of the system. It receives the battery status from the BMS, and it receives a command from the user through the switch. Based on these inputs, it decides what the system should do and sends control signals to the MOSFET and to the indicators. This is also the place where the DQN-based controller will make its charge, idle or discharge decisions once it is loaded on the board in S8.

Switch: The switch gives the user a manual input to the controller. It lets us start or change an operation by hand, which is useful while testing and demonstrating.

LED indication: The controller drives a green LED and a red LED so that the state of the system can be seen at a glance.

3.1.2 Operating Modes

Putting these blocks together, the system works in three ways. In the **charging (G2V)** mode, power flows from the main supply, through the adapter and the DC–DC converter and the BMS, into the battery. In the **discharging (V2G)** mode, the battery feeds power back through the BMS and the converter to the inverter, transformer and loads, which reduces the demand on the supply. In the **idle** mode, the controller keeps the battery disconnected from both, for example when it is almost full, almost empty or protected by the BMS. In every mode, the dashed control signals decide which path is open, and the solid arrows show the power that follows.

3.2 Comparison of Control Approaches

The works above, together with the background studies, show that the decision of when to charge and when to discharge can be taken in several ways compares them in qualitative terms; the quantitative comparison obtained in simulation is given in Chapter 5.

Approach	**How the decision is made**	**Advantage**	**Limitation**
Rule-based (threshold)	Fixed limits on SOC or on the time of day	Simple, predictable, very light	Cannot adapt to changing battery condition or demand
Prediction models (Linear Regression, Decision Tree, Random Forest, XGBoost, LSTM)	Predict the load; a separate rule then acts on the prediction	Capture daily load patterns	Predict only; do not learn the control action
Q-learning	Learns a table of action values	Learns the action from rewards	Table grows with the number of state variables
Deep Q-Network (proposed)	Neural network estimates action values from the full state	Handles several measured inputs; adapts to conditions	Needs training and a model small enough for the ESP32

Table 3.1: Qualitative comparison of V2G decision approaches

Chapter 4

HARDWARE SETUP OF THE PROJECT

4.1 Introduction

This chapter describes the hardware we used to build our Vehicle-to-Grid prototype. The idea behind the setup is simple: a battery stands in for the electric vehicle, a 12 V supply stands in for the grid, and a small fan stands in for the demand. Between them, the system has to watch the battery, decide what to do, and act on that decision safely. To do this, the sensors first measure the battery's voltage, current and temperature. The ESP32 microcontroller then turns these readings into digital values, estimates the State of Charge and checks everything against safe limits. Finally, the relay module, LEDs and display carry out the decision, for example by connecting the battery to the load, stopping the charging or raising an alert when something goes wrong. We chose components that are low-cost, easy to find and safe to handle at 12 V, so that other students can build, modify and repeat the system. The battery is a pack of three 18650 lithium-ion cells in series, and every other component exists to charge it, measure it or protect it. The sections that follow describe each component in turn and explain why we chose it.

4.2 Lithium-ion Battery

The lithium-ion battery is the most important part of the project, because every other component exists to charge it, measure it or protect it. The cells used are 18650 lithium-

ion cells, named after their size of about 18 mm by 65 mm. Each has a nominal voltage of 3.7 V, a full-charge voltage of about 4.2 V, and a rated capacity of 3800 mAh. The two cells sit in a plastic holder so they can be replaced and tested easily.The cell stores energy by moving lithium ions between two electrodes during charging and discharging. This process is not perfectly reversible. With every cycle, a layer called the SEI grows inside the cell and some lithium becomes trapped, so the capacity drops and the internal resistance rises. This gradual decline is measured as State of Health (SOH), the present capacity divided by the original capacity. A value of around 80per is commonly treated as the end of life for an electric vehicle battery.Because lithium-ion cells are sensitive to overcharge, deep discharge and heat, they are never connected directly to the supply or the load. Every connection passes through the relays, sensors and the ESP32 control logic. During the experiments, the cells go through repeated charge and discharge cycles, and the voltage, current and temperature are logged to form the dataset for SOH estimation.

4.3 Microcontroller

The ESP32 is the brain of our V2G prototype. It is a small, low-cost board that combines a dual-core processor, Wi-Fi, Bluetooth, a built-in analog-to-digital converter (ADC) and many input and output pins, so a single board can sense, decide and act. Its work in every cycle can be described in four steps. First, it measures the battery voltage, current and temperature through the sensors, and the ADC turns these analog signals into digital values with 12-bit resolution. Second, it uses these values to calculate the State of Charge (SOC) of the battery. Third, it combines the SOC with the peak or off-peak status of the grid and passes them to the AI model, which selects one of three actions: charge, discharge or idle. Fourth, it carries out the chosen action by switching the relay or MOSFET module, and it shows the voltage, current, SOC and present mode on the OLED display. This loop repeats continuously, so every new decision is based on the latest battery and grid conditions. We chose the ESP32 over simpler boards for a few practical reasons.

It is fast enough to run a small trained TinyML model directly on the board, which means the decision is made locally, without a laptop or an internet connection. It has enough pins to connect the sensors, relay and display at the same time, and it uses little power, which suits an energy management project. Its built-in Wi-Fi also leaves room for future



Figure 4.1: ESP32

work such as remote monitoring and cloud logging. One point to remember is that the ESP32 pins work at 3.3 V and can be damaged by higher voltages, so the sensor signals from the 12 V battery must be scaled down before they reach the board.

4.4 Relay Module

A relay is an electrically operated switch. The idea behind it is simple. The ESP32 can only give small signals at 3.3 V and a few milliamps, but charging, discharging and running a fan need much larger currents.

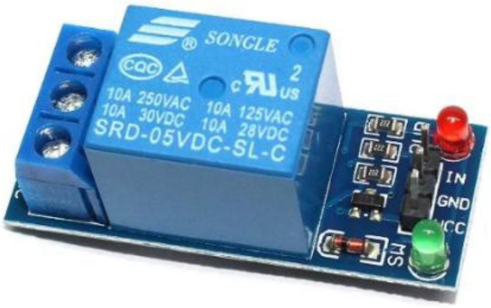


Figure 4.2: Relay Module

A relay module is an electrically operated switch used in our Vehicle-to-Grid (V2G) project to control the charging and discharging of the electric vehicle battery. The ESP32 microcontroller operates at 3.3 V and cannot directly control high-power circuits. There-

fore, the relay module acts as an interface between the low-voltage control circuit and the high-power circuit. Based on the battery voltage and the control signals received from the ESP32, the relay switches the charging or discharging path ON or OFF. When the battery reaches its upper voltage limit during charging, the relay disconnects the charging supply to prevent overcharging. Similarly, it can disconnect the load or discharging path when the battery voltage reaches its lower limit. The relay module helps ensure safe and reliable power flow between the electric vehicle battery and the grid, making it an important component of the V2G system.

4.5 OLED Display

The OLED display functions as a real-time battery monitor on the test bench, displaying live voltage readings, current, temperature, or charging status so users can check cell conditions instantly without a laptop or multimeter. Unlike standard LCDs, every pixel in an OLED produces its own light to deliver sharp, high-contrast text while keeping power consumption low. Specifically, the module features a common 0.96-inch screen with a resolution of 128×64 pixels and connects to the ESP32 via an I2C interface using only four wires (power, ground, clock, and data), minimizing pin usage on crowded boards. This setup aids significantly in troubleshooting by ensuring that any unexpected readings or warning messages are noticed immediately. Unlike a normal LCD, each



Figure 4.3: OLED Display

pixel in an OLED produces its own light, which gives sharp, high-contrast text and keeps power consumption low. The module used here is the common 0.96-inch display with

128 × 64 pixels. It connects to the ESP32 through the I2C interface, which needs only four wires (power, ground, clock and data), so it takes up very few pins on the crowded board. During an experiment, the display also helps with troubleshooting, because an unexpected reading or a warning message is noticed immediately.

4.6 Step-down (Buck) Converter

A step-down or buck converter is basically a clever little module that takes a higher voltage coming from your power adapter and dials it down to a safe, steady level that your battery and electronics can actually handle. Instead of wasting all that extra energy as heat the way old-school linear regulators do, a buck converter relies on a fast-switching internal transistor paired with an inductor and capacitor to smooth everything out into a stable output.



Figure 4.4: Converter

This makes it super efficient—usually hitting between 85 and 95 efficiency—so it runs nice and cool. For this project, the best part is the small built-in potentiometer that lets you fine-tune the output voltage. Because lithium-ion cells are sensitive and need to hit that precise 4.2 V sweet spot to avoid getting dangerously overcharged, being able to dial in that voltage precisely (and double-check it with a multimeter) is an absolute lifesaver.

4.7 Sensors

Voltage Sensor:

The module used is the ACS712, which works on the Hall effect. The current to be measured passes through a copper conductor inside the chip, and the magnetic field it creates is detected by a Hall sensor that produces an output voltage proportional to the current. At zero current the output sits at half the supply voltage (about 2.5 V on a 5 V supply). It moves above this level for current in one direction and below it for the other, so the sensor shows whether the battery is charging or discharging. The measured circuit is also electrically isolated from the output side, which is safer for the ESP32.

The sensor needs some care. Its output change is small for low currents (185 mV per ampere for the 5 A version), so noise and offset can be a large part of the reading. It is therefore important to measure the zero-current offset at start-up and subtract it, to calibrate the sensor against a known current, and to filter the readings. Since the module is supplied at 5 V, its output must be scaled down with a voltage divider before it reaches the 3.3 V pins of the ESP32. The range also matters. The 12 V fan or LED load in this prototype draws only a small current, so the 5 A version gives better resolution than the 20 A or 30 A versions. The sensor needs some care. Its output change is small for low

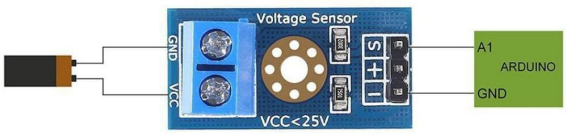


Figure 4.5: Voltage Sensor

currents (185 mV per ampere for the 5 A version), so noise and offset can be a large part of the reading. It is therefore important to measure the zero-current offset at start-up and subtract it, to calibrate the sensor against a known current, and to filter the readings. Since the module is supplied at 5 V, its output must be scaled down with a voltage divider before it reaches the 3.3 V pins of the ESP32. The range also matters. The 12 V fan or LED load in this prototype draws only a small current, so the 5 A version gives better resolution than the 20 A or 30 A versions.

Temperature Sensor:

The temperature sensor monitors the surface temperature of the battery. Temperature is an important safety parameter for Li-ion cells. Charging or discharging at high temperature speeds up ageing and can lead to thermal runaway, while charging at very low temperature can damage the cell permanently. The sensor reading is therefore used both as a safety cut-off and as one of the input features of the AI model, so the controller can choose idle mode when the battery is too hot.



Figure 4.6: Temperature Sensor

The sensor used is the DS18B20 waterproof digital temperature probe. The sensing chip is sealed inside a stainless steel tube with a 100 cm cable, so it can be fixed directly on the battery pack without being affected by moisture. It measures from 55 °C to +125 °C with an accuracy of about ±0.5 °C in the range of 10 °C to +85 °C, which covers the normal working range of a Li-ion battery. It works from a 3.0 V to 5.5 V supply, so it can be powered directly from the 3.3 V pin of the ESP32.

Unlike the voltage and current sensors, the DS18B20 gives a digital output. It communicates over the 1-Wire protocol, which needs only a single data line, and the result is already in degrees Celsius with a resolution of up to 12 bits. This avoids ADC errors and noise. Each sensor also has a unique 64-bit address, so several probes could be placed on one data line to monitor different parts of the battery. The probe has three wires: red for VCC, black for ground and white for data. A 4.7 k pull-up resistor must be connected between the data line and VCC for reliable communication. At the highest resolution a conversion takes about 750 ms, which is fast enough for battery temperature, since it changes slowly.

Chapter 5

SYSTEM DEVELOPMENT AND IMPLEMENTATION

5.1 Introduction

This chapter describes the hardware prototype built to demonstrate AI-assisted V2G operation on a safe, low-voltage platform. A single 3.3 V lithium cell stands in for the EV battery, and a low-voltage DC power supply stands in for the grid. The ESP32 is the heart of the system. It reads the battery voltage and current, estimates the state of charge (SOC), runs the decision model, and controls the switching circuit that moves energy between the battery and the grid side. A small DC load represents the demand on the grid.

We chose a low-voltage setup for three reasons. First, it is safe, so students can build and test the circuit without any risk of electric shock. Second, the parts are cheap and easy to find. Third, the 3.3 V cell matches the ESP32's own logic level, which keeps the interfacing simple. Most importantly, the key V2G decision of when to charge and when to discharge does not depend on the voltage level, so the control idea can be demonstrated fully on this small platform. The component names given here are proposed for the prototype, and the final part numbers and ratings will be confirmed against their data sheets once the setup is complete.

5.2 System Development Methodology

The prototype was developed in a series of practical steps: choosing the hardware, designing the circuit, interfacing the components, programming the ESP32, and finally integrating everything into one working unit. Each component was chosen for what it does in the system and for how well it works with the others, especially with the 3.3 V logic of the ESP32. Sensors for battery voltage, current and temperature are connected to the ESP32, which reads their values continuously. The microcontroller processes these readings, estimates the state of charge, and shows the results on an OLED display. A web-based dashboard was also developed, so the battery status can be monitored wirelessly from a mobile phone or a computer.

5.3 Hardware Component Selection

The hardware components are selected according to the requirements of the battery monitoring system. Each component performs a specific function in collecting, processing or displaying battery information.

The major components used in the prototype are:

- ESP32 development board
- Lithium-ion battery
- Battery Management System (BMS)
- Voltage sensor module
- Fuse
- ACS712 current sensor
- Temperature sensor
- OLED display
- Relay module
- DC fan as an external load

- DC to DC LM2596 buck converter
- Dual-Watt LED Strip
- Connecting wires

5.4 Circuit Design and Hardware Assembly

The prototype works like a small-scale version of a vehicle-to-grid setup. Mains power is stepped down by a 12 V adapter and fed through a DC-DC buck charger, which brings it to 12.6 V to charge a 3S pack of 18650 Li-ion cells, with a BMS guarding the cells against overcharge, deep discharge and imbalance. When energy needs to flow the other way, the battery's output passes through an ACS712 current sensor into a DC-AC inverter with a step-up transformer, producing 230 V AC. A changeover relay then decides whether the bulb and fan are supplied from the grid or from the battery-backed inverter. At the center of it all is the ESP32, powered at 5 V through an LM2596 buck converter. It reads current from the ACS712 and battery voltage from the voltage sensor, then uses that information to switch the relay and to drive a DC motor load through a MOSFET module with PWM. Two LEDs show the system status, and a push switch allows manual control, so together the pieces demonstrate energy moving between grid, battery and load under intelligent control.

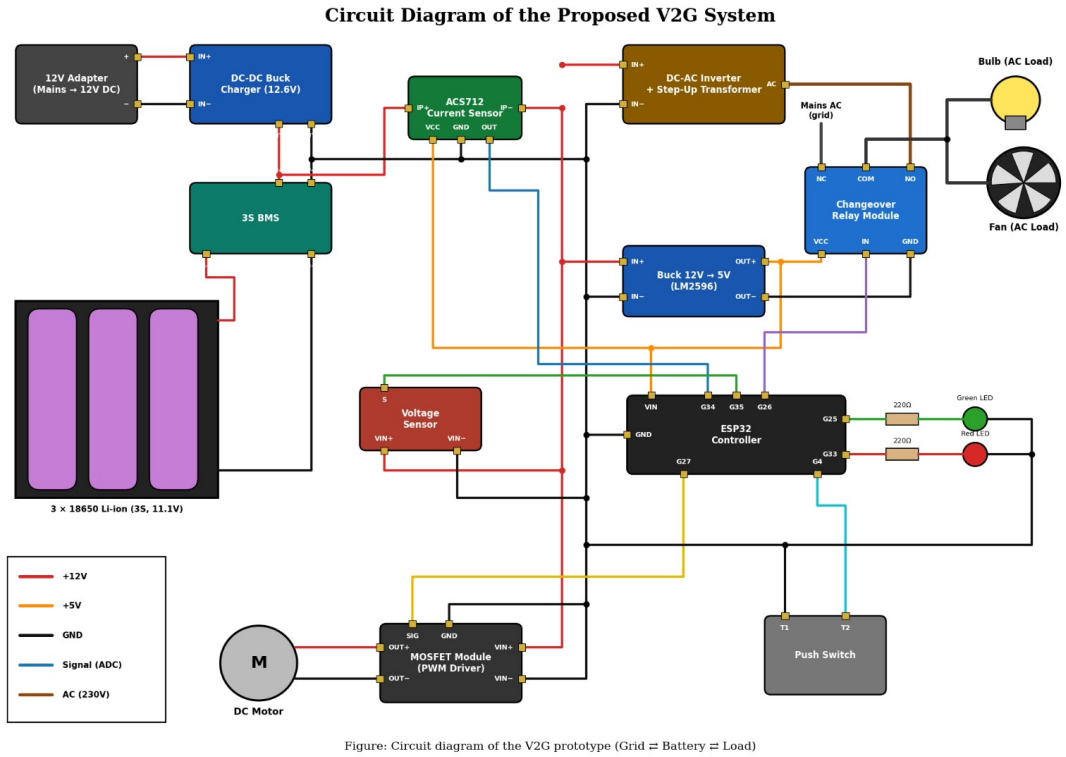


Figure: Circuit diagram of the V2G prototype (Grid ⇄ Battery ⇄ Load)

Figure 5.1: Circuit Diagram of the Proposed V2G System.

5.5 Sensor Interfacing and Data Acquisition

In the proposed Vehicle-to-Grid (V2G) energy management system, sensors are used to monitor the battery parameters during operation. These sensors are connected to the ESP32 microcontroller, which collects and processes the measured data. The collected information helps in monitoring the battery condition and understanding the power flow in the system. Voltage, current, and temperature are the main parameters considered for monitoring. The measured values can be displayed on a monitoring interface to observe the system's performance in real time.

5.5.1 Voltage Measurement

A voltage sensor module is connected to the battery circuit to measure its output voltage. The sensor provides a scaled voltage signal to the analog input of the ESP32. The controller processes this signal to calculate the corresponding battery voltage.

5.5.2 Current Measurement

An ACS712 current sensor is used to measure the current flowing through the battery circuit. When the DC fan is operated as an external load, the sensor measures the current drawn by the load. The measured current value is processed by the ESP32 and displayed on the monitoring interface.

5.5.3 Temperature Measurement

Temperature monitoring helps observe the thermal condition of the system during operation. A temperature sensor is connected to the ESP32 to measure the temperature at the selected location near the battery or other relevant components. The ESP32 collects the sensor readings at regular intervals, allowing temperature variations to be monitored. If a DHT sensor is used, it can also measure the surrounding humidity. These measurements provide additional information about the operating environment and help identify unusual temperature changes during the experiment.

5.6 ESP32 Programming and Control Logic Development

The ESP32 is programmed using the Arduino programming environment. The program is developed to collect sensor readings, process the measured values and update the connected display and web dashboard.

The main operations performed by the ESP32 program include:

- Reading the voltage sensor output.
- Measuring current using the ACS712 sensor.
- Collecting temperature readings.
- Processing the sensor measurements.
- Updating the OLED display.
- Establishing Wi-Fi communication.
- Hosting the web-based monitoring interface.

- Controlling the relay connected to the DC fan.

The ESP32 continuously performs these operations to provide updated information about the battery parameters.

5.7 OLED Display Integration

The 0.96-inch OLED display is connected to the ESP32 over I2C, which needs only four wires. It shows the battery voltage, current, temperature and SOC, together with the present operating mode, so the system can be observed directly on the prototype without a laptop or a multimeter. The green and red LEDs give a quick indication of the mode and of faults, and the buzzer sounds when a safety limit is crossed. A separate digital voltmeter across the battery terminals is used to cross-check the voltage sensor.

5.8 Load Testing Using a DC Fan

A 12 V DC fan is connected to the grid side of the prototype through a relay module. It acts as the demand on the simulated grid, like the appliances and homes that draw power from the real grid. We use it to see how the battery and the supply behave when the load is switched on and off.

When the fan is switched on, it draws current from the supply, and from the battery when the battery is discharging to support it. The voltage sensor measures the battery voltage, the ACS712 sensor measures the current drawn, and the DS18B20 probe reports the battery temperature. The ESP32 reads these values, works out the State of Charge, and shows them on the OLED display. Running the fan makes the voltage dip and the current rise, and these changes can be compared with the readings taken when no load is connected. Switching the fan on also gives us a simple way to create a peak-demand condition for the AI model.

5.9 Integration of Hardware and Software

After building and checking each module on its own, we combined everything into the complete prototype. The 12 V Li-ion battery pack and the 12 V DC supply form the

power side. The voltage, current and temperature sensors feed their readings to the ESP32, and the relay module sits between the supply, the battery and the load to control the direction of power flow.

On the software side, the ESP32 program reads the sensors, calculates the SOC from the voltage and the current, and passes these values, together with the demand condition, to the trained AI model. The model predicts whether the battery should charge, discharge or stay idle, and the ESP32 then switches the relay accordingly. Basic safety limits for overtemperature, low voltage and full charge are checked before any command is sent, so the battery is protected even if the model suggests otherwise. The OLED display shows the live voltage, current, SOC, temperature and operating mode. Together, the hardware and software form a closed loop that measures, decides, acts and measures again.

5.10 Testing and Performance Evaluation

The prototype was tested step by step. First, each sensor was checked separately. The voltage sensor readings were compared with a multimeter and corrected by calibration. The ACS712 zero-current offset was measured and subtracted, and its readings were checked against a known load current. The temperature readings were confirmed at room temperature and while the battery warmed slightly under load.

Next, the relay switching was tested with the fan as the load. The relay connected the battery to the fan, switched the connection after a few seconds and repeated the cycle continuously, which confirmed that the prototype can switch repeatedly between the intended operating states.

The DQN controller was evaluated separately in simulation over a 24-hour cycle, where it kept the net load lower and more stable than the rule-based and Q-learning controllers in the peak window. Testing the controller on the hardware, together with the safety limits under different battery and demand conditions, is planned for S8.

5.11 Prototype Development and Final Assembly

The hardware demonstration models the charging and discharging path of a vehicle-to-grid (V2G) system on a small scale. The AC mains supply is first converted to 12 V

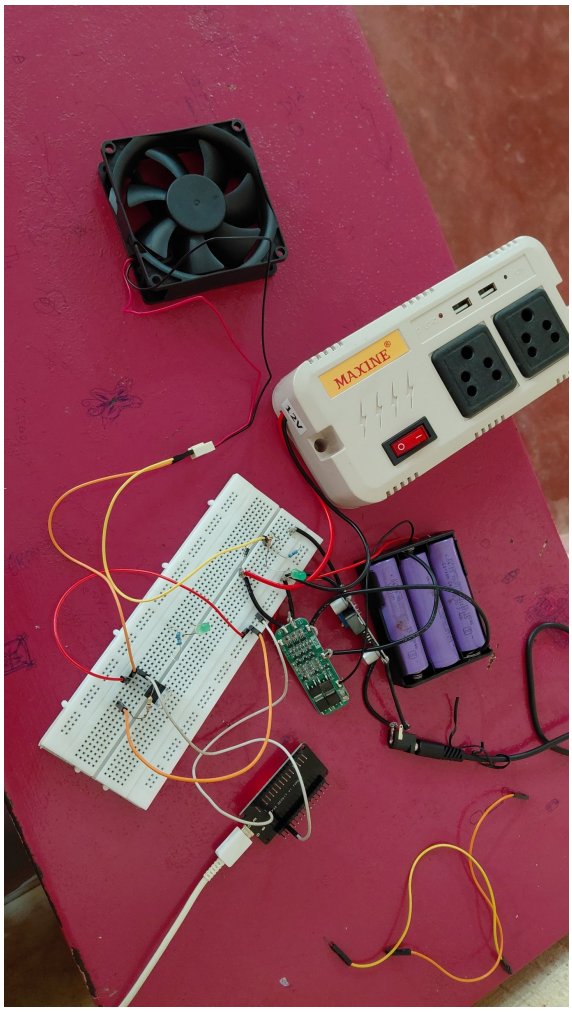


Figure 5.2: Developed Hardware Prototype

DC by an adapter. This is stepped down and regulated by a DC-DC buck converter to a suitable charging voltage and passed through a 3S battery management system (BMS) module. The BMS protects the battery pack from overcharge, over-discharge and short circuit while charging three 3.7 V Li-ion cells connected in series. An ESP32 microcontroller controls the entire system and decides when the charging and discharging operations take place. A MOSFET is used as an electronic relay: the ESP32 drives its gate, and it switches the cooling fan on while the battery is charging, which shows that the charging path is active and keeps the circuit cool. In the discharging mode, the stored energy of the battery pack is supplied as 12 V DC to a DC-to-AC inverter, which converts it to 230 V AC and delivers it to the output socket. This represents the vehicle battery supplying power back to the grid or to a local load. An LED indicator shows the status of the circuit, so the user can see when the system is active. Overall, the prototype demonstrates bidirectional energy flow between the grid and the battery, with the ESP32

providing intelligent control and the BMS providing protection.

5.12 System Operation

The V2G prototype works by monitoring the battery's voltage, current, temperature, and State of Charge (SOC) using sensors connected to the ESP32. The controller processes these readings and sends the required data to the trained AI model, which helps select the appropriate operating mode: charging, discharging, or idle. The relay module controls the power connections according to the selected mode, while a 12 V DC fan is used to represent the electrical load. The OLED display shows the battery parameters and current operating mode. Safety limits are also included to help protect the battery during operation. Overall, the system demonstrates how AI-based control can manage battery power flow according to changing load conditions.

5.13 Summary

This chapter described the development of the V2G prototype. The ESP32 acts as the main controller: it reads the voltage, current and temperature sensors, estimates the SOC, applies the safety limits and operates the relays that connect the battery to the supply, to the load or to neither. A 12 V DC fan is used as the load to create changing demand, and the OLED display and LEDs show the battery condition and operating mode. The hardware assembly, sensor interfacing and relay switching were completed and tested in S7, and the DQN controller was developed and compared in simulation. The remaining work is to run the trained controller on the ESP32, add the bidirectional converter and build the live dashboard in S8.

Chapter 6

PROJECT STATUS COMPLETED IN S7

6.1 Hardware Prototype Development

The initial hardware prototype of the proposed battery monitoring and energy management system was developed. The main objective of this stage was to establish a working hardware setup that could monitor the operating parameters of a battery and provide a foundation for implementing intelligent battery management features.

The prototype was assembled on a perforated PCB board, with an ESP32 microcontroller serving as the central processing unit. A battery pack was connected to the circuit to provide the required power supply. Different electronic modules were integrated with the ESP32 to enable battery parameter monitoring and control operations.

The hardware connections were completed, and the individual components were checked to ensure proper interfacing. The initial prototype provided a platform for further development and testing of the proposed system.

6.1.1 Integration of ESP32

The ESP32 microcontroller was selected as the main controller because of its processing capabilities, multiple GPIO pins and built-in Wi-Fi connectivity. It acts as the central unit responsible for collecting sensor data and controlling the connected hardware components.

The ESP32 also provides the communication interface required for transferring the collected information to the monitoring platform. The integration of the microcontroller with the different modules was completed, and the basic data acquisition process was successfully tested.

6.1.2 Hardware Testing and Initial Verification

The hardware testing was carried out to check the basic operation of the V2G prototype. A DC fan was connected as the grid load, and the battery was connected to the circuit through a relay module. When the system was switched on, the relay controlled the connection, allowing the battery to supply power to the fan. After a few seconds, the relay switched the connection, and the process repeated continuously. This helped us observe the relay switching operation and the fan's response during each cycle. Through this initial testing, we verified the basic working of the hardware components and confirmed that the prototype could demonstrate repeated switching between the intended operating states.

6.2 Simulation and Algorithm Comparison

Compares the V2G control and load prediction algorithms over a 24-hour cycle, with the proposed DQN controller highlighted. The prediction algorithms follow the actual net power pattern, while the DQN controller selects charging, idle or discharging actions from the observed system state. In the plot, the actual net grid load reaches about 9–11 kW during hours 9 to 12, whereas the DQN-controlled load stays at about 3.5–4.7 kW in the same hours. This indicates the peak-hour load reduction that the project aims to achieve

6.2.1 DQN Training Convergence and Reward Improvement

This graph shows how the DQN agent improved during training. The x-axis is the training episode (one simulated day each, 800 in total), and the y-axis is the mean reward, averaged over a sliding window of 25 episodes to smooth out noise.

Because the reward is built from penalties (peak overload, battery use, switching, and

missing the end-of-day SOC target), values closer to zero are better. The curve starts at about -23, rises steeply during the first 300 episodes as the agent explores and learns, then gradually flattens near -5 after about episode 500. This flattening shows the learning has stabilized, meaning the agent has found a consistent charging and discharging policy. Small ripples remain because training days vary and the agent still takes occasional random actions (epsilon stays at a minimum of 0.05).

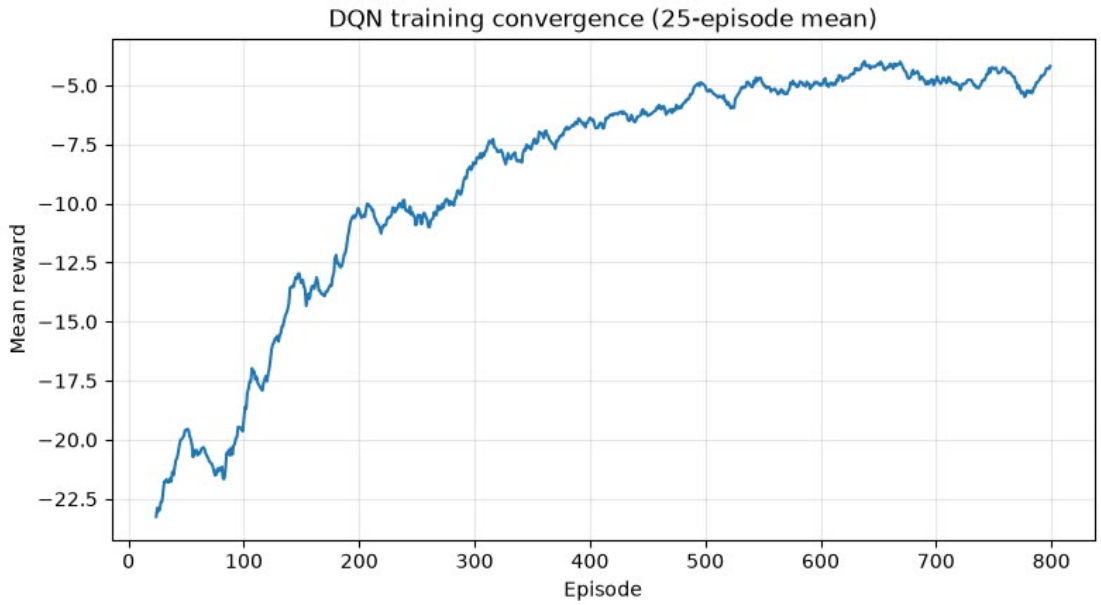


Figure 6.1: Mean Reward Progression During DQN Training

6.2.2 Comparison plot

This graph compares the net grid load over a 24-hour test cycle for four cases: the baseline with no V2G, a rule-based controller, tabular Q-learning, and the Deep Q-Network (DQN). Without V2G, the load peaks at about 11.7 kW between hours 9 and 12 and is also high at the start of the day (about 9.2 kW at hour 0). The shaded bands mark these high-demand hours. During this window the DQN agent reduces the load to about 4.2-4.7 kW, a cut of more than 60 percentage at the peak, by discharging the vehicle batteries into the grid. The rule-based controller also lowers the load at hour 9 but is inconsistent, with sharp spikes at hours 3, 17 and 20 (up to about 9.7 kW), because fixed thresholds cannot adapt to changing conditions. Q-learning follows the DQN closely in the early hours but shows larger swings in the afternoon and evening, such as the peak near 7.9 kW at hour 18. The DQN keeps the load mostly between 3 and 6 kW through the day,

giving the smoothest profile of the three methods. One exception is the rise to about 8.5 kW at hour 23, which likely comes from charging the vehicles late in the night. Overall, the results show that the DQN is the most effective at peak shaving and load flattening.

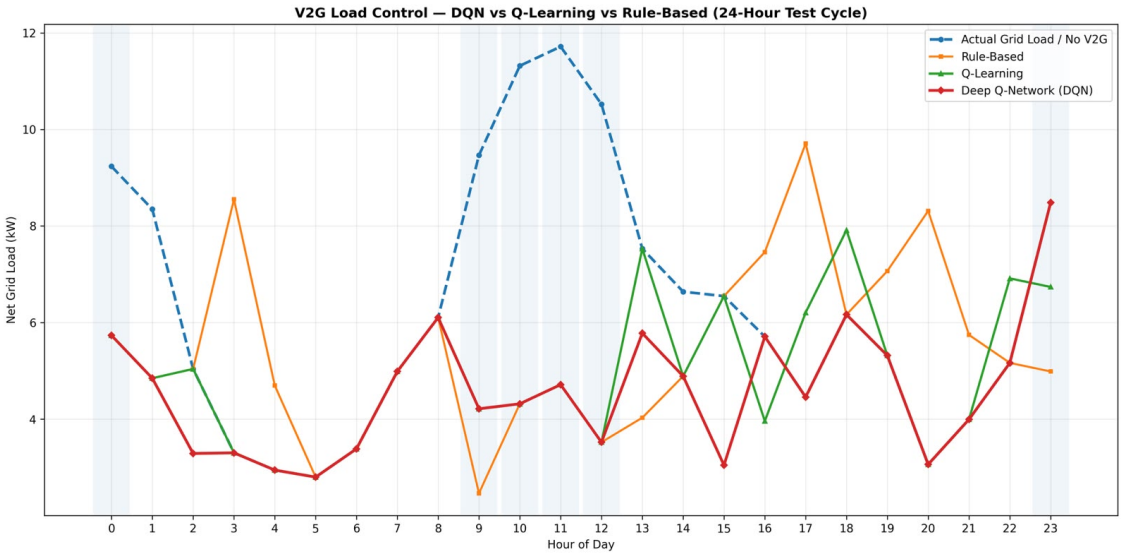


Figure 6.2: V2G Load Control–DQN vs Q-Learning vs Rule-Based

Chapter 7

PROJECT TO BE COMPLETED IN S8

7.1 Introduction

Our project will be further developed to improve its functionality, efficiency, and monitoring capabilities. The current prototype demonstrates basic battery charging and fan operation using a relay module. As the next step, additional sensors will be integrated to measure battery voltage, charging and discharging current, battery status, and power flow. These measurements will help monitor the system and improve the control of energy transfer.

The work done so far has given us a working prototype and a DQN controller that performs well in simulation. Even so, we see this as a starting point rather than the end of the project. The prototype can sense the battery and switch its power paths, but the trained controller does not yet run on the board, the monitoring is still limited to the OLED display, and the support the battery gives to the grid has so far been shown only in simulation. This chapter describes the work we plan to do next: studying two more reinforcement learning algorithms, building a live dashboard, and demonstrating peak shaving on the hardware itself, along with some wider improvements that could follow.

7.2 Study and Analysis of DDPG and PPO Algorithms

Our DQN controller chooses one of three fixed actions: charge, idle or discharge. This is simple and works well, but it is also a limitation. A real V2G system does not only decide whether to charge or discharge; it also decides how much power to exchange. With only three choices, the controller either exchanges full power or none, which can be harsh on the battery and gives less smooth control of the grid load. DQN is also known for being sensitive to training settings and needing many episodes to settle. For these reasons we want to study two other algorithms that are widely used in energy management research, DDPG and PPO, and see whether they improve on our current controller.

7.2.1 Deep Deterministic Policy Gradient (DDPG)

DDPG is an actor-critic method built for continuous actions. It uses two neural networks. The actor looks at the current state and directly outputs an action, which in our case could be a charging or discharging power between a negative and a positive limit. The critic then judges how good that action was. Like DQN, it uses experience replay and target networks to keep training stable, and it adds a little noise to the actions during training so that the agent keeps exploring. The attraction of DDPG for V2G is that it can produce a smooth power command instead of a switch between three modes. Its drawbacks are that it can be sensitive to hyperparameters and sometimes becomes unstable during training, so it has to be tuned carefully.

7.2.2 Proximal Policy Optimization (PPO)

PPO is also an actor-critic method, but it learns in a different way. Instead of reusing old experience from a replay buffer, it collects fresh experience, updates the policy, and limits how far each update can move the policy from the previous one. This clipping is what makes PPO known for stable and reliable training, and it is often a good first choice when training is difficult. PPO can work with discrete actions as well as continuous ones, so it can be tested on exactly the same charge, idle and discharge problem that DQN solves, and then extended to continuous power. The price of this stability is that PPO usually needs more interaction data than methods that reuse old experience.

7.2.3 Plan for the comparison

To keep the comparison fair, we will train DQN, DDPG and PPO on the same V2G dataset and in the same simulated environment, with the same state (SOC, grid load, EV power demand, battery temperature and time) and the same reward idea. The three controllers will then be compared on the points that matter for our project..

We will look at how much each controller reduces the peak load, how well it keeps the SOC within safe limits, how quickly and steadily it learns, and how much unnecessary charging and discharging it causes. Since the final controller has to run on a small microcontroller, we will also check how large each trained network is. For DDPG and PPO only the actor network is needed at run time, which helps keep the model small enough for TinyML on the ESP32. Whichever algorithm gives the best balance of performance, stability and size will be the one we take forward to the hardware. It is also possible that the original DQN remains the best choice for a three-mode prototype, and that would be a useful finding in itself.

7.3 Dashboard Making and Displaying

At present the only way to see what the prototype is doing is to look at the small OLED screen. That is fine for the person standing next to it, but it does not show trends, and it is hard to follow during a demonstration. A dashboard will let us watch the system from a phone or a laptop and see how the battery responds over time, which also makes it much easier to find problems and explain the results.

The dashboard will display the live battery voltage, current, temperature and SOC, the present operating mode (charging, discharging or idle), the direction of power flow between the supply, battery and load, and the energy used. A short history chart of these values will show how they change when the fan load is switched on and off. A clear warning will appear when a safety limit is crossed, and the action chosen by the controller will be shown next to the readings.

7.4 Peak Shaving Using Hardware

Peak shaving means using stored energy to supply the load during hours of high demand, so that less power is drawn from the grid at the busiest time, and recharging the battery when demand is low. We have already shown this effect in simulation, where the DQN controller held the net load well below the actual load in the peak window. The goal now is to show it on the real hardware, because a result that can be measured on a real circuit is more convincing than a plot alone.

We will record the supply current, battery current, battery voltage, SOC and temperature during several peak and off-peak cycles. From these we can calculate how much the supply current falls during the peak, how far the SOC drops, and whether the battery stays inside its safe limits. The first version will use simple rules to decide when to discharge. Once the trained controller runs on the ESP32, the same test will be repeated with the AI deciding, so that the rule-based and AI-based results can be compared on real hardware. As in every other test, the safety limits remain in charge, and the system goes to idle if the battery becomes too empty or too warm.

7.5 Final Testing, Validation and Project Demonstration

After all the planned improvements are in place, we will test the complete system to check that the hardware, the controller and the dashboard work properly together. We will watch the battery charging process, the sensor readings, the relay operation and the dashboard information under different operating conditions, such as peak and off-peak demand created by switching the load on and off. We will also test the Grid-to-Vehicle and Vehicle-to-Grid functions once the power-conversion and control systems are ready, and compare the hardware results with the simulation results to see how closely the real system follows the controller we trained. If any problems turn up during testing, we will study them, correct them wherever possible and note them honestly, because knowing where the system falls short is as useful as showing where it works. Finally, we will demonstrate the project, explain how it works step by step and present the results we have measured, including the reduction in peak demand. This last stage will show us how well the system performs and point out what can be improved in future development.

7.6 Expected Outcome

At the end of S8, the project is expected to deliver a improved V2G prototype that does more than switch a battery on and off: one that watches its own condition, decides for itself and shows what it is doing. The comparison of DQN, DDPG and PPO should tell us which algorithm gives the best balance of peak-load reduction, safe battery operation, stable learning and small model size, and the winner will be loaded on the ESP32 so that the decision is made on the board. A live dashboard on a phone or laptop should let anyone follow the voltage, current, temperature, SOC, operating mode and power flow as they change, which will make testing and demonstrations much clearer. We also expect to show peak shaving on the real hardware, where the battery supports the load during a simulated peak and recharges when demand falls, with the drop in supply current measured on the circuit instead of only plotted from simulation. Along the way, we hope to gain a practical understanding of how energy moves between the grid and the battery and how stored energy can be managed wisely. Altogether, the project aims to deliver an affordable, easy-to-repeat platform that can support further work on AI-based smart grid and electric-vehicle charging applications.

7.7 Mapping with POs and PSOs

The project is mapped with the Programme Outcomes (POs) and the Programme Specific Outcomes (PSOs) in Table **??**.

Mapping levels: 1 – Low, 2 – Medium, 3 – High.

Table 7.1: Mapping of the project with POs and PSOs

PO / PSO	**Level**	**Justification**
PO1: Engineering Knowledge	3	Electronics, embedded systems, battery fundamentals and machine learning applied to V2G control.
PO2: Problem Analysis	3	Reviewed the literature and identified the limits of fixed-rule V2G control.
PO3: Design/Development of Solutions	3	Designed a safe low-voltage prototype and a DQN controller for charge, idle and discharge decisions.
PO4: Conduct Investigations of Complex Problems	2	Compared prediction models, Q-learning and DQN over a 24-hour cycle and tested the sensors and relay switching.
PO5: Engineering Tool Usage	3	Used Python, MATLAB/Simulink, Proteus, the ESP32 platform and sensor modules.
PO6: The Engineer and the World	3	Supports peak-load reduction and better use of idle EV batteries.
PO7: Ethics	2	Sources are cited; battery-protection limits and low-voltage operation keep the work safe.
PO8: Individual and Collaborative Team Work	2	Work was shared among the four members.
PO9: Communication	2	Report and presentation to the evaluation panel.
PO10: Project Management and Finance	2	Low-cost components chosen and a Semester VIII work plan prepared.
PO11: Life-Long Learning	2	Learnt reinforcement learning, TinyML and sensor interfacing.

PSO	Level	Justification
PSO1: Analysis and design of electronic circuits and embedded architectures	3	Sensor interfacing, switching circuit and ESP32-based control designed.
PSO2: Solving problems with hardware, software and modern tools	3	Hardware prototype and AI software combined with safety limits.
PSO3: Innovation in emerging areas with ethics and societal needs	2	AI-based V2G control addressing the energy needs of EV users and the grid.

7.8 Mapping with SDGs

Table 7.2: Mapping of the project with SDGs

SDG	Contribution of the project
SDG 7: Affordable and Clean Energy	Uses idle EV battery energy to support the grid at peak hours; the platform can be extended with solar power.
SDG 9: Industry, Innovation and Infrastructure	Provides an affordable AI-based smart-grid platform that supports innovation.
SDG 11: Sustainable Cities and Communities	Eases the evening peak caused by EV charging activities.
SDG 13: Climate Action	Supports electric mobility and the use of renewable energy by making the grid more stable.

Bibliography

[1] M. S. Hashim, J. Y. Yong, V. K. Ramachandaramurthy, K. M. Tan, M. Mansor, and M. Tariq, "Priority-Based Vehicle-to-Grid Scheduling for Minimization of Power Grid Load Variance," *Journal of Energy Storage,* vol. 39, Art. no. 102607, 2021.

[2] A. R. Singh, R. Seshu Kumar, K. R. Madhavi, F. Alsaif, M. Bajaj, and I. Zaitsev, "Optimizing Demand Response and Load Balancing in Smart EV Charging Networks Using AI Integrated Blockchain Framework," *Scientific Reports,* vol. 14, Art. no. 31768, 2024.

[3] A. Nouri, A. Lachheb, and L. El Amraoui, "Optimizing Efficiency of Vehicle-to-Grid System with Intelligent Management and ANN-PSO Algorithm for Battery Electric Vehicles," *Electric Power Systems Research,* vol. 226, Art. no. 109936, 2024.

[4] M.-J. Jang and E. Oh, "Deep-Reinforcement-Learning-Based Vehicle-to-Grid Operation Strategies for Managing Solar Power Generation Forecast Errors," *Sustainability,*vol. 16, no. 9, Art. no. 3851, 2024.

[5] R. P. Upputuri and B. Subudhi, "A Comprehensive Review and Performance Evaluation of Bidirectional Charger Topologies for V2G/G2V Operations in EV Applications," *IEEE Transactions on Transportation Electrification* vol. 10, no. 1, pp. 583–595, Mar. 2024,

APPENDIX

Code: DQN training convergence.

```
"""Graph 1: DQN training convergence. Needs artifacts/training_rewar
import argparse
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

parser = argparse.ArgumentParser()
parser.add_argument("--artifacts", default="artifacts")
args = parser.parse_args()
out = Path(args.artifacts)

rewards = np.loadtxt(out / "training_rewards.csv", delimiter=",", sk
window = min(25, len(rewards))
moving = np.convolve(rewards, np.ones(window) / window, mode="valid"

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.plot(np.arange(len(moving)) + window - 1, moving, color="tab:blue
ax.set(title="DQN training convergence (25-episode mean)", xlabel="E
ax.grid(alpha=0.3)
fig.tight_layout()
```

```
fig.savefig(out / "plot1_training_convergence.png", dpi=180)
plt.close(fig)
print("saved", out / "plot1_training_convergence.png")
```

Code: Comparison

```
import numpy as np
import matplotlib.pyplot as plt

# 24-hour simulation
hours = np.arange(24)

# 1. Actual Grid Load (Without V2G)
actual_load = [
    9.2, 8.35, 5.05, np.nan, np.nan, np.nan,
    np.nan, np.nan, 6.1, 9.45, 11.3, 11.7,
    10.5, 7.55, 6.65, 6.55, 5.7,
    np.nan, np.nan, np.nan, np.nan, np.nan,
    np.nan, np.nan
]

# 2. Rule-Based Control
rule_based = [
    np.nan, np.nan, 5.05, 8.55, 4.7, 2.8,
    3.4, 5.0, 6.1, 2.5, 4.3, 4.7,
    3.5, 4.0, 4.9, 6.55, 7.45, 9.7,
    6.15, 7.05, 8.3, 5.75, 5.2, 5.0
]
```

```
# 3. Q-Learning Control
q_learning = [
    np.nan, 4.85, 5.05, 3.3, np.nan, np.nan,
    np.nan, np.nan, np.nan, np.nan, np.nan,
    np.nan, np.nan, 7.55, 4.9, 6.55, 3.95,
    6.2, 7.9, 5.3, np.nan, np.nan, 6.9, 6.75
]

# 4. Deep Q-Network (DQN)
dqn = [
    5.75, 4.85, 3.3, 3.3, 2.95, 2.8,
    3.4, 5.0, 6.1, 4.2, 4.3, 4.7,
    3.5, 5.8, 4.9, 3.05, 5.7, 4.45,
    6.15, 5.3, 3.05, 4.0, 5.2, 8.5
]

# Create graph
plt.figure(figsize=(18, 8))

# Highlight operating periods
shaded_regions = [
    (0, 0.45),
    (9, 9.45),
    (10, 10.45),
    (11, 11.45),
    (12, 12.45),
    (23, 23.45)
]

for start, end in shaded_regions:
    plt.axvspan(
```

```
        start - 0.45, end,
        color='lightblue',
        alpha=0.20
    )

# Plot actual grid load
plt.plot(
    hours, actual_load,
    color='tab:blue',
    linestyle='--',
    marker='o',
    linewidth=3,
    label='Actual Grid Load / No V2G'
)

# Plot Rule-Based
plt.plot(
    hours, rule_based,
    color='tab:orange',
    marker='s',
    linewidth=2.2,
    label='Rule-Based'
)

# Plot Q-Learning
plt.plot(
    hours, q_learning,
    color='tab:green',
    marker='^',
    linewidth=2.5,
    label='Q-Learning'
)
```

```
# Plot DQN
plt.plot(
    hours, dqn,
    color='red',
    marker='D',
    linewidth=3,
    label='Deep Q-Network (DQN)',
    zorder=5
)

# Title
plt.title(
    'V2G Load Control | DQN vs Q-Learning vs Rule-Based (24-Hour Tes
    fontsize=16,
    fontweight='bold'
)

# Labels
plt.xlabel('Hour of Day', fontsize=12)
plt.ylabel('Net Grid Load (kW)', fontsize=12)

# Axis limits
plt.xticks(hours)
plt.xlim(-1.5, 24.5)
plt.ylim(2, 12.2)

# Grid and legend
plt.grid(True, alpha=0.25)
plt.legend(loc='upper right', fontsize=11)

plt.tight_layout()
```

```
# Display
plt.show()
```



Department of Electronics and Communication Engineering
Vidya Academy of Science & Technology
Thalakkottukara, Thrissur - 680 501
(http://www.vidyaacademy.ac.in)